



Domain Driven Design - Java

Prof. Gilberto Alexandre das Neves
profgilberto.neves@fiap.com.br

Stream API

A **Stream API** foi introduzida no **Java 8** e é usada para **processar coleções de dados** (como listas e conjuntos) de forma **declarativa** (ou seja, dizendo o que fazer, e não como fazer).

Ela permite:

- Filtrar dados (**filter**)
- Transformar dados (**map**)
- Agrupar (**collect**)
- Ordenar (**sorted**)
- Contar, somar, encontrar máximos/mínimos, etc.

Tudo isso com um estilo funcional, mais limpo e conciso.

O **Collection Framework** do Java inclui classes como **List**, **Set**, **Map** etc. A **Stream API** não substitui essas classes, mas permite processar os dados que elas contêm de forma eficiente.

Você pode pensar na **Stream** como um **pipeline**: os dados entram por um lado, passam por uma série de operações (filtros, transformações, etc.) e saem do outro lado processados.

Características das **Streams**:

- **Não armazenam dados**: uma stream não é uma coleção, ela só processa os dados da coleção.
- **Imutáveis**: os dados originais não são alterados.
- **Pipelines**: permitem encadear operações.

```
7 ▶ public class FiltroSemStream {
8 ▶     public static void main(String[] args) {
9         ArrayList<String> heróis = new ArrayList<>(Arrays.asList("Homem
        Aranha", "Wolverine", "Hulk", "Capitão América", "Homem
        Elástico", "Pantera Negra", "Viúva Negra", "Mulher Maravilha",
        "Homem de Ferro", "Miss Marvel", "Mulher Invisível", "Cíclope"));
10     ArrayList<String> heróisComH = new ArrayList<>();
11
12     for (String herói : heróis) {
13         if (herói.startsWith("H")) {
14             heróisComH.add(herói);
15         }
16     }
17
18     JOptionPane.showMessageDialog(null, String.format("Heróis que
        começam com a letra H:\n%s", heróisComH), "Heróis com H",
        JOptionPane.WARNING_MESSAGE);
19 }
20 }
```

```
9  ▶ public class FiltroComStream {
10 ▶     public static void main(String[] args) {
11         ArrayList<String> heróis = new ArrayList<>(Arrays.asList("Homem
            Aranha", "Wolverine", "Hulk", "Capitão América", "Homem
            Elástico", "Pantera Negra", "Viúva Negra", "Mulher Maravilha",
            "Homem de Ferro", "Miss Marvel", "Mulher Invisível", "Cíclope"));
12
13         List<String> heróisComH = heróis.stream()
14             .filter(herói -> herói.startsWith("H"))
15             .collect(Collectors.toList());
16
17         JOptionPane.showMessageDialog(null, String.format("Heróis que
            começam com a letra H:\n%s", heróisComH), "Heróis com H",
            JOptionPane.WARNING_MESSAGE);
18     }
19 }
```

Praticando...

Implemente a classe **PokemonHashMap** com o método **main**. Veja a seguir as orientações:

1. Peça para o usuário digitar o nome e o tipo de pokémons e armazene em um objeto da classe **HashMap** (armazene somente pokémons novos, não permita repetições). Quando quiser parar ele deve digitar “**fim**”.
2. Peça para o usuário digitar um **tipo** qualquer a escolha dele.
3. Percorra todo o **HashMap** em busca de todos os nomes de pokémons do tipo escolhido pelo usuário. Exiba na tela o **tipo** que ele escolheu e o **nome de todos** os pokémons daquele tipo.
4. Perguntar ao usuário se deseja continuar (em caso afirmativo repetir os passos de 1 a 4 novamente).
5. Em caso negativo, exibir uma mensagem se despedindo do usuário e encerre o programa.


```
7 ▶ public class PokemonHashMap {
8 ▶     public static void main(String[] args) {
9         HashMap<String, String> pokemons = new HashMap<>();
10        do {
11            try {
12                String nome, tipo;
13                do {
14                    nome = JOptionPane.showInputDialog("Digite o nome de um Pokémon ou digite\n\"fim\" para encerrar").toUpperCase();
15                    if (!nome.equals("FIM")) {
16                        tipo = JOptionPane.showInputDialog("Digite o tipo do Pokémon informado anteriormente").toUpperCase();
17                        if (pokemons.containsKey(nome)) {
18                            JOptionPane.showMessageDialog(null, "Este Pokémon já foi cadastrado!",
19                                "Atenção", JOptionPane.WARNING_MESSAGE);
20                        } else {
21                            pokemons.put(nome, tipo);
22                        }
23                    } while (!nome.equals("FIM"));
24                }
25            } catch (Exception e) {
26                e.printStackTrace();
27            }
28        } while (true);
29    }
30 }
```

```
24 String escolha = JOptionPane.showInputDialog("Digite um tipo qualquer a sua
    escolha").toUpperCase();
25 String nomesEncontrados = "";
26 for (Map.Entry<String, String> entrada : pokemons.entrySet()) {
27     if (entrada.getValue().equals(escolha)) {
28         nomesEncontrados += entrada.getKey() + "\n";
29     }
30 }
31 JOptionPane.showMessageDialog(null, String.format("Para o tipo: %s foram
    encontrado(s) o(s) pokémon(s):\n%s", escolha, nomesEncontrados), "Pokémons",
    JOptionPane.INFORMATION_MESSAGE);
32 } catch (Exception e) {
33     JOptionPane.showMessageDialog(null, e.getMessage(), "Erro", JOptionPane
        .ERROR_MESSAGE);
34 }
35 } while (JOptionPane.showConfirmDialog(null, "Deseja continuar?", "Atenção", JOptionPane
    .YES_NO_OPTION, JOptionPane.QUESTION_MESSAGE) == 0);
36 JOptionPane.showMessageDialog(null, "Fim de Programa!", "Adeus", JOptionPane
    .WARNING_MESSAGE);
37 }
38 }
```

Reimplemente a classe `PokemonHashMap` com o nome de **`PokemonComStream`**. Altere o código para utilizar a **Stream API** na aplicação do **filtro** para a busca dos nomes dos pokémons de um tipo escolhido pelo usuário. Orientações:

1. Peça para o usuário digitar o nome e o tipo de pokémons e armazene em um objeto da classe **HashMap** (armazene somente pokémons novos, não permita repetições). Quando quiser parar ele deve digitar “**fim**”.
2. Peça para o usuário digitar um **tipo** qualquer a escolha dele.
3. Utilizando **Stream API** busque todos os nomes de pokémons do tipo escolhido pelo usuário. Exiba na tela o **tipo** que ele escolheu e o **nome de todos** os pokémons daquele tipo.
4. Perguntar ao usuário se deseja continuar (em caso afirmativo repetir os passos de 1 a 4 novamente).
5. Em caso negativo, exibir uma mensagem se despedindo do usuário e encerre o programa.



Java como programar. Paul Deitel e Harvey Deitel. Pearson, 2011.

Java 8 – Ensino Didático : Desenvolvimento e Implementação de Aplicações. Sérgio Furgeri. Editora Érica, 2015.

Até breve!